

Coding-Scheme Benchmark for Spiking Neural Networks

A Research Log and Technical Report

Comparison of Rate, TTFS, and Phase Coding on EMNIST and CIFAR-10

Sean Moon

Table of Contents

1. Research Overview
2. Literature / Paper List
3. Experimental Setup
4. Experiments
5. Results
6. Contribution
7. What I Learned / Observations
8. Limitations
9. Future Work

1. Research Overview

This project investigates how the effectiveness of temporal coding schemes in spiking neural networks depends on dataset characteristics and the choice of evaluation metric, not on the coding scheme in isolation.

1.1 Research Questions

- **RQ1:** Which coding scheme (Rate, TTFS, or Phase) performs best on static image datasets, and does the answer depend on whether accuracy, computational cost, or latency defines “best”?
- **RQ2:** For phase coding specifically, how do oscillation frequency (cycle_steps) and phase resolution (num_bins) affect performance, and do these temporal degrees of freedom interact?
- **RQ3:** How do coding-scheme behavior, tuned hyperparameters, and sensitivity patterns change between EMNIST and CIFAR-10?

1.2 Motivation

Spiking neural networks (SNNs) represent information in spike timing rather than only spike rate, which underpins their claimed energy and latency advantages over conventional rate-coded networks and ANNs. Some existing SNN coding comparisons use a common or fixed output readout across different input coding schemes. In this project, each input coding scheme is paired with a corresponding scheme-specific readout (rate→rate, TTFS→TTFS, phase→phase) so that the implemented pipelines can be evaluated consistently. Because mismatched encoding–readout pairings were not tested, this report does not claim that matched pairings are superior; a direct matched-versus-mismatched comparison is left as future work.

1.3 Overall Hypothesis

No single coding scheme dominates across all evaluation axes. Instead, each scheme occupies a different point on an accuracy/energy/latency trade-off surface, and which point is “best” depends on (a) the deployment objective (accuracy-critical vs. energy-constrained vs. latency-critical) and (b) properties of the dataset itself (e.g., how bimodal its pixel-intensity distribution is, how many classes it has, whether it is a clean handwritten-character set or a natural image set). A secondary hypothesis specific to phase coding is that its two temporal degrees of freedom (oscillation frequency and phase resolution) interact, so that sweeping either one alone (holding the other fixed) will not find the true joint optimum.

1.4 What Was Compared

- Three neural coding schemes: Rate coding, Time-to-First-Spike (TTFS) coding, and Phase (phase-of-firing) coding; see section 3 for full definitions.
- Each scheme with a matched readout (coding == readout for every result in section 5), under an otherwise identical single-hidden-layer LIF network, training protocol, and time budget (section 3.1).
- Two static image datasets: EMNIST (Balanced split) and CIFAR-10 (section 3.3).

- For phase coding specifically: oscillation frequency (`cycle_steps`), phase resolution (`num_bins`), and their joint interaction (section 4, section 5.4–5.7).
- *Not compared in this phase of the project: Burst coding and event-based (DVS) datasets appear in the literature reviewed in section 2 and were not implemented or run under this project's fairness and matched-encoding-readout conventions (section 1.6); a related, earlier project has exploratory implementations of both, not yet re-validated under those conventions (see section 8 Limitations and section 9 Future Work).*

1.5 Why These Datasets Were Selected

EMNIST (Balanced, 47 classes). A canonical static-image benchmark in the SNN coding literature (Table 2A/2B). Its pixel intensities are close to bimodal (a clean background versus a handwritten stroke), which makes it a favorable case for threshold-based temporal codes (TTFS, phase), since a single threshold cleanly separates “fires” from “doesn't fire.”

CIFAR-10 (10 classes, 3-channel natural images). Deliberately chosen as a harder, less favorable case: its pixel intensities are spread broadly with no clear background/foreground bimodality, so it tests whether conclusions drawn on EMNIST (e.g., the phase-coding threshold and (`num_bins`, `cycle_steps`) optimum) transfer to a dataset with different statistics, or whether they must be retuned per dataset (section 5.2 answers this: they must be retuned).

1.6 Why These Metrics Were Selected

Accuracy alone does not capture what makes a coding scheme attractive for spiking hardware, so every experiment in section 4–5 reports four metrics together:

- **test_acc:** classification accuracy, selected for reporting only after the hyperparameter itself was chosen on validation accuracy (never on test accuracy), to avoid leakage.
- **avg_input_spikes / avg_hidden_spikes:** raw spike counts at the two layers where they can be counted.
- **avg_energy_cost / avg_synops:** a fan-out-weighted synaptic-operations proxy, not just a spike count. Input spikes fan out across `hidden_dim` = 512 synapses, while hidden spikes fan out to only `num_classes` synapses; since fan-out-weighted cost dominates, a scheme that looks “expensive” by raw hidden-spike count can still be cheap overall if its input-spike count is low (this is exactly the TTFS/phase pattern in section 5.1).
- **avg_latency:** mean decision latency: for each sample, the earliest timestep after which the network's running arg-max prediction stays equal to its final-timestep prediction for the rest of the sequence. This measures how early a correct-or-not decision has effectively stabilized, independent of which coding scheme produced it.

2. Literature / Paper List

Papers reviewed during this project, grouped into three categories. Author/year identifications and topical descriptions were compiled by the project owner; full citation details are not yet attached and

should be added before this list is used in a written report or thesis. “Relevance to this project” is this report’s own assessment of why each entry matters here, not a claim about the paper’s findings; a “Main finding” column is intentionally left for the reader to fill in after (re-)reading each paper, rather than summarized secondhand.

Paper	Year	Datasets used	Coding schemes	Relevance to this project
Guo et al.	2021	MNIST, Fashion-MNIST, CIFAR-10	Rate, TTFS, Phase, Burst	Primary baseline for multi-scheme coding comparison; overlaps this project’s dataset choices and metric set (accuracy, latency, SynOps, hardware cost).
Park et al.	2020	MNIST, CIFAR-10, CIFAR-100	Rate, TTFS, Phase, Burst	A second, independent multi-coding-scheme benchmark, useful for cross-checking whether trends found here (e.g. rate’s accuracy edge, TTFS’s efficiency edge, section 5.1) generalize across implementations.
Kim et al.	2022	MNIST, Fashion-MNIST, CIFAR-10	Rate, TTFS (direct coding)	Directly relevant to the accuracy–efficiency trade-off framing in section 1.6 and section 5.1, on the same family of datasets used broadly in this literature.
Niu et al.	2023	MNIST, Fashion-MNIST, CIFAR-10, DVS128, etc.	Rate, TTFS, Phase, Burst	Survey giving the broader taxonomy this project’s Rate/TTFS/Phase scope sits inside; also background on the DVS128 (event-based) domain this project did not experimentally cover (section 8).
Deng et al.	2020	CIFAR-10, CIFAR-100	Rate	Background on SNN-vs-ANN computational efficiency on CIFAR-10/100; relevant context for why this project’s CIFAR-10 accuracies plateau well below EMNIST’s (section 5.2, section 8).
Eshraghian et al.	2021	MNIST, Fashion-MNIST, CIFAR-10	Rate	Reference for the surrogate-gradient BPTT training methodology used throughout this project’s LIF model (section 3.1).

Table 2B: Phase Coding–Specific Literature

Paper	Year	Domain	Phase assumption / main idea	Relevance to this project
O’Keefe & Recce	1993	Biological recordings (hippocampus)	Theta oscillation; spike timing shifts relative to oscillation phase	Biological origin of phase coding; motivates it as a bio-plausible temporal code rather than only an engineering convenience.

Paper	Year	Domain	Phase assumption / main idea	Relevance to this project
Lisman & Jensen	2013	Neuroscience theory	Multi-frequency (theta–gamma) coupled oscillations multiplex information	Background for why this project treats phase frequency (cycle_steps) and resolution (num_bins) as two separate, jointly-interacting degrees of freedom (section 4, section 5.5–5.6).
Zhang et al.	2020	Synthetic temporal signals	Temporal delay represented via phase/frequency relationships	Background for interpreting the frequency × resolution interaction effect found in this project (section 5.6).
Hwang & Kung	2021	CIFAR, ImageNet	Single-spike phase representation for ANN→SNN conversion	Directly relevant: applies a phase-style single-spike code to static natural images, the same static-image setting this project uses for phase coding.
Guo et al.	2021	MNIST, Fashion-MNIST	Coding schemes differ in temporal behavior	Also appears in Table 2A; here specifically as a phase-vs-other-coding temporal-behavior comparison.
Zhang et al.	2022	Temporal / signal tasks	Time-frequency phase structure; geometric interpretation	Theoretical grounding for why a frequency × resolution interaction (rather than two independent effects) is plausible.
Murray et al.	2023	Synthetic / RNN tasks	Phase-like codes can emerge from recurrent oscillations without being hand-designed	Relevant to the not-yet-completed learnable-frequency direction (section 4 Experiment 4, section 9); motivates the differentiable relaxation prototype built in this project.
He et al.	2023	CIFAR-10/100, DVS-Gesture, SHD	Adaptive oscillatory phase coding	Closest published analogue to this project's phase-coding parameter sweeps (section 4); also covers event-based datasets this project did not test.

Table 2C: DVS / Event-Based Literature (Supporting References)

Source	Why reviewed / main observation	Connection to this project
Event-Driven Classifier for SNNs	DVS sensors provide native temporal events, unlike synthetically time-encoded static frames.	Explains why static and event-based data should not be treated identically when applying a temporal code.
CIFAR10-DVS	The sensor generates events from temporal contrast changes, not from a fixed encoding rule.	DVS input is not the same kind of object as an artificially phase/TTFS-encoded static image.
Temporal Pulses Driven SNN	Native event timing can be exploited directly by the network.	Suggests additional temporal encoding (rate/TTFS/phase) may be unnecessary, or must be designed differently, for event-based input.

Source	Why reviewed / main observation	Connection to this project
DVS-Attacks	DVS events carry (x, y, polarity, timestamp); timing is already part of the raw data.	Confirms DVS data has inherent temporal structure that this project's static-image encoders were not designed to exploit.

Observation from the DVS literature

Unlike static datasets, DVS data already contain temporal information in the event timestamps. Applying an additional temporal encoding scheme such as phase coding to already-temporal event data is therefore not directly comparable to applying phase coding to static images, where the encoder is the only source of timing information. This is why DVS literature was treated as supporting background (Table 2C) rather than as the experimental basis of this phase of the project, and why extending this benchmark to a genuinely event-based dataset is listed as future work (section 9) rather than folded into the current static-image results.

3. Experimental Setup

This section is written to be carried into a Methods section largely as-is.

3.1 Model

3.1.1 LIF Neuron

The hidden layer is a single fully-connected leaky integrate-and-fire (LIF) layer trained with a surrogate gradient. At each timestep t :

- Synaptic current: $\mathbf{I}_t = \mathbf{W} \cdot \mathbf{x}_t$ (a linear layer over the input spikes at that timestep).
- Membrane update: $\mathbf{v}_t = \mathbf{v}_{t-1} + (\mathbf{I}_t - \mathbf{v}_{t-1}) / \tau$, with $\tau = 2.0$.
- Spike: $\mathbf{s}_t = \mathbf{H}(\mathbf{v}_t - \mathbf{v}_{th})$, with $\mathbf{v}_{th} = 0.1$, where $\mathbf{H}$ is a hard Heaviside in the forward pass.
- Surrogate backward pass: the gradient of $\mathbf{H}$ is replaced by the derivative of a sigmoid, $\sigma(\mathbf{v}_t - \mathbf{v}_{th}) \cdot (1 - \sigma(\mathbf{v}_t - \mathbf{v}_{th}))$, so gradients can flow through spiking nonlinearities during backpropagation-through-time (BPTT).
- Reset: hard reset to 0 on spike, $\mathbf{v}_t \leftarrow \mathbf{v}_t \cdot (1 - \mathbf{s}_t)$.

$\mathbf{v}_{th} = 0.1$ and $\tau = 2.0$ are held identical across every coding scheme and every experiment in this report, on purpose, so that accuracy differences between schemes are attributable to encoding/readout choices, not to different hidden-layer dynamics.

3.1.2 Network Architecture

image $[B, C, H, W] \rightarrow$ encoder $\rightarrow$ input spike train $[T, B, N] \rightarrow$ LIF hidden layer $[T, B, 512] \rightarrow$ linear output layer $\rightarrow$ leaky-integrated output current sequence $[T, B, \text{num_classes}] \rightarrow$ scheme-specific readout $\rightarrow$ logits $[B, \text{num_classes}]$.

The output layer is a plain linear projection of the hidden spikes at each timestep, passed through a second leaky integrator ($\tau_{\text{out}} = 2.0$, no spiking nonlinearity) rather than a second spiking layer; it produces an analog “output current” trace over time, which the readout function (section 3.2) then pools into a single class score per sample. `hidden_dim = 512` for every experiment; `time_steps (T) = 32`.

3.1.3 Training

- Optimizer: Adam, learning rate $1e-3$.
- Loss: cross-entropy on the readout logits.
- Epochs: 35 per run.
- Batch size: 128.
- Gradient flow: full BPTT through all $T = 32$ timesteps via the surrogate gradient (section 3.1.1).
- Model/hyperparameter selection: `best_val_acc` across epochs, never test accuracy; test accuracy is read off only after the epoch/hyperparameter choice is fixed.
- Seeds: 3 seeds (0, 1, 2) for most experiments; single-seed for a few exploratory sweeps, explicitly flagged in section 4–5 wherever that applies.

3.2 Coding Schemes

3.2.1 Rate Coding

Encoding principle: Pixel intensity is interpreted directly as a per-timestep firing probability; brighter pixels fire more often. Spikes are drawn independently at every timestep (Bernoulli by default, optional Poisson variant).

Parameters: `rate_scale` (default 0.5) scales pixel intensity before it is used as a firing probability, capped at 1.0 (a knob for spike budget independent of accuracy, section 5.8).

Implementation: `encoders/rate.py`: $p = \text{clamp}(\text{rate_scale} \cdot x, 0, 1)$; spike at timestep t with probability p , drawn fresh each timestep.

Temporal behavior/readout: Spikes are spread roughly uniformly across the whole $T = 32$ window; readout sums spike counts over time (`readouts/rate.py`), the classic “count up the spikes” decoder.

3.2.2 Time-to-First-Spike (TTFS) Coding

Encoding principle: Each pixel emits at most one spike; higher-intensity pixels fire earlier, lower-intensity pixels fire later, and pixels below a threshold never fire.

Parameters: `threshold` (minimum normalized intensity to fire at all; project default 0.2 on EMNIST, 0.3 on CIFAR-10, see section 5.9); `gamma` (latency-mapping exponent, default 1.0, linear).

Implementation: `encoders/ttfs.py`: $t = t_{\text{min}} + (t_{\text{max}} - t_{\text{min}}) \cdot (1 - x)^{\text{gamma}}$, rounded to the nearest integer timestep; pixels with x below threshold never spike.

Temporal behavior/readout: At most one spike per input unit, so `avg_input_spikes` is far lower than rate coding for the same image. Readout (`readouts/ttfs.py`) applies an exponential early-time-weighted sum

over the output current sequence, so earlier spikes count for more; the decay rate is scaled to $4.0 / T$ so the weighting spans the full window regardless of T .

3.2.3 Phase (Phase-of-Firing) Coding

Encoding principle: Pixel intensity is mapped to a phase bin within a periodic oscillation cycle: brighter pixels fire at an earlier phase, darker pixels at a later phase; pixels below a threshold never fire. Spikes recur every cycle unless `repeat_cycles = False`, in which case each pixel fires at most once, during its first cycle only.

Parameters: `num_bins` (phase resolution: how many distinct phase offsets intensity is quantized into); `cycle_steps` (oscillation period, in timesteps: smaller = higher relative frequency); `threshold`; `repeat_cycles` (default `False`, project-wide, section 3.3 rationale below).

Implementation: `encoders/phase.py`: `phase_bin = floor((1 - x) · (num_bins - 1))`; `offset = round(phase_bin / (num_bins - 1) × (cycle_steps - 1))`; spike emitted at $t_k = \text{offset} + k \cdot \text{cycle_steps}$ for each valid cycle k .

Temporal behavior/readout: Readout (`readouts/phase.py`) applies a cosine weighting keyed to the same period: $\text{weight}(t) = \cos(2\pi \cdot (t \bmod \text{period}) / \text{period})$. Because `repeat_cycles = False` means no new input arrives after $t = \text{cycle_steps}$, a windowing option zeroes the weight for $t \geq \text{cycle_steps}$ (“coasting tail”) so the cosine pattern doesn't keep contributing a period-dependent residual after the encoding has stopped (section 5.7).

Burst coding appears throughout the literature reviewed in section 2 (Guo et al. 2021, Park et al. 2020, Niu et al. 2023) as a fourth scheme alongside Rate/TTFS/Phase. It is not implemented in this codebase and has no results in this report under this project's methodology; a related, earlier project has a working burst encoder that has not yet been re-validated under this project's fairness conventions (see section 8 Limitations and section 9 Future Work).

3.3 Datasets

3.3.1 EMNIST (Balanced split)

- Type: static grayscale image, 28×28, handwritten characters/digits.
- Classes: 47 (the Balanced split's merged upper/lowercase + digit set).
- Preprocessing: `ToTensor`, then `Normalize(mean = 0.5, std = 0.5)` → pixel values in $[-1, 1]$; encoders internally map this back to $[0, 1]$ before applying their firing rule.
- Train/val/test split: official torchvision EMNIST train/test split, with the official training set further split 90% train / 10% validation (seeded by the run's seed, so each seed sees a different train/val partition).
- Temporal conversion: none inherent to the dataset; all timing is introduced by the encoder (section 3.2); $T = 32$ timesteps for every EMNIST experiment.

3.3.2 CIFAR-10

- Type: static RGB natural image, 32×32×3.

- Classes: 10.
- Preprocessing: ToTensor, then Normalize(mean = (0.5,0.5,0.5), std = (0.5,0.5,0.5)).
- Train/val/test split: official torchvision CIFAR-10 train/test split, with the official training set further split 85% train / 15% validation (seeded).
- Temporal conversion: none inherent, same as EMNIST; flattened $3 \times 32 \times 32 = 3,072$ -dim input, $T = 32$ timesteps.

3.3.3 DVS-Gesture (not used)

DVS-Gesture and other event-based datasets appear in the literature reviewed (section 2, Table 2B/2C) but were not obtained or run in this project. As Table 2C's observation notes, event-based data already carries native spike timing, which raises a design question (how, or whether, to re-encode already-temporal data with rate/TTFS/phase) that this project's static-image encoders do not address. A related, earlier project has a working DVS-GC loader and preliminary rate/TTFS/phase/burst results on it, but not under this project's fairness and matched-encoding-readout conventions (section 1.6). This is the largest single gap between the literature surveyed and the experiments actually run under this project's methodology; see section 8 and 9.

4. Experiments

Experiments in the order they were run. Each entry gives its purpose, its result, and (where relevant) what decision it fed into.

Experiment 1: Baseline Coding Comparison

Purpose: Compare Rate, TTFS, and Phase coding under the exact same network, training protocol, and time budget, each with its own matched readout and its own separately-tuned threshold/resolution (the fairness convention of section 1.6).

Setting: EMNIST, 3 seeds per scheme. Phase uses its joint-grid-optimal ($\text{num_bins} = 4$, $\text{cycle_steps} = 8$) found in Experiment 3b (this table therefore reflects the final, already-tuned configuration; see section 5.1 for full numeric results and interpretation).

Experiment 2: Oscillation Frequency Sweep

Purpose: Investigate RQ2: with phase resolution fixed ($\text{num_bins} = 8$), does varying the oscillation period $\text{cycle_steps} \in \{4, 6, 8, 12, 16\}$ change accuracy?

Caveat found during this sweep: $\text{cycle_steps} < \text{num_bins}$ makes phase bins collide onto the same offset slot within a cycle, confounding the frequency effect at the smallest cycle_steps values; this motivated Experiment 3b's collision-free joint grid. Run at a single seed only (see section 5.4).

Experiment 3: Phase Resolution Sweep

Purpose: Investigate RQ2 from the other axis: with oscillation period fixed ($\text{cycle_steps} = 16$), does varying phase resolution $\text{num_bins} \in \{4, 8, 12, 16\}$ change accuracy, independent of frequency?

Result (section 5.5): accuracy decreases monotonically as num_bins increases; coarser phase binning wins at this fixed period, which is the first hint that resolution and frequency are not independent.

Experiment 3b: Joint Frequency $\times$ Resolution Grid

Purpose: Run every valid (num_bins , cycle_steps) combination with $\text{cycle_steps} \geq \text{num_bins}$ (20 cells, 3 seeds each) to check for an interaction effect that Experiments 2 and 3 could not reveal on their own, since each varies only one axis while fixing the other. This experiment directly tests the interaction question posed in RQ2.

Result (section 5.6): ($\text{num_bins} = 4$, $\text{cycle_steps} = 8$) is a clear outlier, well above the best cell found by either single-axis sweep. This combination became the default phase setting used in Experiment 1 and every later phase-coding experiment on EMNIST.

Experiment 3c: Phase-Neighborhood Bug-Check

Purpose: Before trusting the (4, 8) outlier from Experiment 3b, verify it is a genuine local maximum rather than an isolated bug artifact by densely sampling its neighborhood ($\text{num_bins} \in \{3, 4, 5\}$, $\text{cycle_steps} \in \{6, 7, 8, 9, 10\}$, 3 seeds) and checking whether nearby cells form a smooth hill.

Run twice: once with the normal phase readout, once with the “coasting-tail” cosine weighting zeroed out (phase_windowed), to separately check whether the effect might instead be an artifact of cycle_steps being a clean divisor of the readout window. Result (section 5.7): (4, 8) sits inside a genuine smooth local hill, not an isolated spike; confirmed as a real effect, not a bug.

Experiment 4: Threshold Sweeps (TTFS, Phase)

Purpose: Because threshold determines which pixels fire at all, it directly trades accuracy against spike count/energy for both TTFS and phase coding. Swept threshold $\in \{0.05, 0.1, 0.15, 0.2, 0.25, 0.3\}$ (EMNIST, plus a wider $\{0.0\text{--}0.5\}$ range on CIFAR-10, whose pixel intensities are not cleanly bimodal).

Decision rule: pick the threshold with the best validation accuracy, then check its position on the energy curve. On EMNIST, both TTFS and phase land on threshold = 0.2 (section 5.9). On CIFAR-10, phase accuracy falls off sharply and monotonically as threshold increases, so threshold = 0.0 (no filtering) is used instead.

Experiment 5: Rate-Scale Sweep

Purpose: Rate coding has no threshold knob; $\text{rate_scale} \in \{0.3, 0.5, 0.7, 0.9, 1.0\}$ was swept as its analogous accuracy/energy knob, scaling how large a firing probability a fully-bright pixel gets.

Result (section 5.9): accuracy is roughly flat (~81–82%) across the whole range, while spike count, energy cost, and latency all change substantially; useful as a near-free energy/latency dial for rate coding.

Experiment 6: repeat_cycles Ablation

Purpose: Test whether letting phase-encoded pixels re-fire every cycle for the whole window (repeat_cycles = True), instead of only in their first cycle (False), is worth its extra spike cost.

Result (section 5.9): True gives a small accuracy edge (+0.1–0.8 pp) at roughly 33% higher avg_synops, driven by firing 4× more input spikes, which fan out to hidden_dim = 512 synapses versus hidden spikes fanning out to only num_classes. False is the efficiency-rational default used everywhere else in this report.

Experiment 7: CIFAR-10 Baseline, Threshold, and Joint Grid

Purpose: Re-run the baseline comparison (Experiment 1), the threshold sweeps (Experiment 4), and the joint frequency × resolution grid (Experiment 3b) on CIFAR-10, to test whether EMNIST's conclusions and tuned hyperparameters transfer to a dataset with a very different pixel-intensity distribution.

Result (section 5.2): the qualitative ranking of schemes is preserved, but the specific tuned values are not: CIFAR-10's phase optimum is (num_bins = 4, cycle_steps = 4, threshold = 0.0), not EMNIST's (4, 8, 0.2). This is direct evidence that these hyperparameters are dataset-specific and must be retuned per dataset.

Experiment 8: Learnable Oscillation Frequency (Proof-of-Concept)

Purpose: The discrete phase encoder gives cycle_steps zero gradient by construction (spike placement is integer indexing, which PyTorch cannot differentiate through), so every result above found cycle_steps by grid search, not by training. This diagnostic checks whether a differentiable relaxation could instead let cycle_steps be learned end-to-end.

Method: a soft, Gaussian-relaxed spike-placement variant used during training (with a discrete snap-back at evaluation time) is compared against the hard/discrete encoder in a standalone script (experiments/learnable_frequency_diagnostic.py). Result (section 5.8): the hard variant's cycle_steps never moves (zero gradient, as expected); the soft variant shows a non-zero gradient and cycle_steps drifting downward step-over-step, confirming a learnable version is mechanically feasible. Not yet integrated into the main training pipeline (section 9 item 3).

5. Results

5.1 Main Results: EMNIST Baseline

results/baseline_final_results.csv, mean of 3 seeds, SynOps = avg_energy_cost, latency in timesteps out of T = 32.

Coding	Setting	Accuracy	Input spikes	Hidden spikes	SynOps	Latency
Rate	—	81.7%	2,201.6	853.0	1,167,332	16.79
TTFS	th = 0.2	70.3%	171.3	1,706.1	167,876	30.14

Coding	Setting	Accuracy	Input spikes	Hidden spikes	SynOps	Latency
Phase	nb=4, cs=8, th=0.2	77.5%	171.3	15,029.2	794,059	7.72

Rate coding achieved the highest accuracy (81.7%) but at roughly 7× TTFS's synaptic cost and about double its latency. TTFS is the cheapest scheme in terms of the SynOps proxy (approximately 4.7× cheaper than phase and 7.0× cheaper than rate) but also the least accurate and the slowest to settle on a decision, using nearly the full 32-step time budget. Phase coding sits in between on both accuracy (+7.2 pp over TTFS) and energy, but wins decisively on latency: under 8 steps to a stable decision versus 17–30 for the other two. No scheme wins on all three axes, which directly supports the hypothesis in section 1.3: the “best” coding scheme is a function of which axis matters for the deployment target: accuracy-critical (rate), energy-constrained (TTFS), or latency-critical (phase).

5.2 Dataset-Dependent Results: EMNIST vs. CIFAR-10

results/cifar10_baseline_results.csv, mean of 3 seeds.

Coding	Setting	Accuracy	Input spikes	SynOps	Latency
Rate	—	46.5%	23,424.7	12,005,804	23.91
TTFS	th = 0.3	44.0%	2,219.2	1,152,848	30.90
Phase	nb=4, cs=4, th=0.0	45.5%	3,064.2	1,636,518	16.44

EMNIST

- Accuracies are high (70–82%) and well separated between schemes, with a stable, interpretable trade-off: rate is most accurate, TTFS is cheapest, phase is fastest.
- Its near-bimodal background-vs-stroke pixel distribution makes threshold-based schemes (TTFS, phase) behave predictably: threshold sweeps (section 5.9) show wide, flat plateaus around the chosen value (0.2).
- The phase-coding frequency × resolution interaction (section 5.6) is large and reproducible here: a 4-point accuracy gap between the tuned setting and the next-best cell.

CIFAR-10

- All three schemes plateau in a much narrower, lower band (44–47%); the ranking is preserved (rate > phase > TTFS) but the gaps between schemes shrink substantially compared to EMNIST.
- Pixel intensities are not cleanly bimodal, so phase accuracy is far more threshold-sensitive here: it falls off sharply and monotonically as threshold increases (section 5.9), unlike EMNIST's flat plateau.
- The tuned phase optimum shifts from (4, 8) on EMNIST to (4, 4) on CIFAR-10 (section 5.6): the same num_bins, a different cycle_steps.
- Same architecture, same training protocol, same time budget: the low absolute accuracy is best attributed to the model itself (a single flattened-input LIF layer, no convolutional structure) being a weak backbone for 32×32×3 natural images, not to any one coding scheme (section 8).

The relative performance of coding schemes was not consistent across datasets in its details (tuned hyperparameters, sensitivity to threshold, size of the frequency×resolution effect), even though the

coarse accuracy ranking held. This is the central dataset-dependence argument of this project: coding efficiency should be interpreted in relation to dataset characteristics, not treated as a fixed property of the coding scheme alone.

5.3 Sensitivity Analysis: Phase Frequency × Resolution

This is the most consequential sensitivity result in the project (also section 4 Experiments 2–3c). Single-axis views first:

- **Frequency alone** (num_bins = 8 fixed, single seed, results/frequency_results.csv): best at cycle_steps = 6 (72.2%); cycle_steps = 4 is collision-confounded and lowest (64.8%).
- **Resolution alone** (cycle_steps = 16 fixed, 3 seeds, results/resolution_results.csv): accuracy decreases monotonically as num_bins increases: 4 bins (73.4%) beats 16 bins (69.8%).

Joint grid (all 20 valid (num_bins, cycle_steps) cells with cycle_steps ≥ num_bins, 3 seeds each, results/frequency_resolution_grid_results.csv):

(num_bins, cycle_steps)	Test accuracy
(4, 8)	77.5%
(4, 12)	73.7%
(4, 16)	73.4%
(4, 4)	73.3%
(4, 6)	73.0%
(8, 4)	72.9%

(4, 8) is a ~4 pp outlier above the next-best cell, and above either single-axis sweep's best result, an interaction neither single-axis experiment could reveal alone. The phase-neighborhood bug-check (section 4 Experiment 3c, results/phase_neighborhood_results.csv) confirmed this is a genuine smooth local maximum (num_bins = 4 with cycle_steps in 7–10 all scoring 77–78%), not an isolated artifact.

On CIFAR-10 (results/cifar10_frequency_resolution_grid_results.csv), the joint-grid winner is a different cell entirely:

(num_bins, cycle_steps)	Test accuracy
(4, 4)	45.5%
(4, 6)	43.9%
(12, 4)	43.0%
(8, 6)	43.0%
(8, 4)	42.6%

Which parameters strongly affect accuracy: num_bins (resolution) has the largest and most consistent effect of the two phase parameters on both datasets; low num_bins (4) wins in every joint-grid table above. Which parameters affect spike count: cycle_steps has a larger effect on avg_hidden_spikes than num_bins does (longer periods before repeat_cycles kicks in produce more decision-window overlap), while threshold (section 5.9 below) is the dominant lever for avg_input_spikes on both TTFS and phase.

Which schemes are stable/unstable: TTFS is the most threshold-stable scheme on EMNIST (near-flat accuracy across threshold = 0.05–0.3, results/threshold_ttfs_results.csv) but the least stable on CIFAR-10 by comparison to phase's steep sensitivity there; phase is the least stable scheme overall with respect to its own hyperparameters, on either dataset.

5.4 Threshold and Rate-Scale Sensitivity

EMNIST threshold sweeps (single seed; note this table's num_bins/cycle_steps differ slightly between the TTFS and phase runs from the values used in section 5.1's tuned baseline; see results/threshold_ttfs_results.csv, results/threshold_phase_results.csv):

Threshold	TTFS accuracy (EMNIST)	Phase accuracy (EMNIST)
0.10	78.0%	71.2%
0.15	77.9%	71.2%
0.20	77.8%	71.8%
0.25	—	72.1%
0.30	77.4%	71.9%

Both schemes are nearly flat across this range on EMNIST; threshold = 0.2 was chosen as a reasonable, round middle point rather than because it is a sharp optimum. Rate coding has no threshold, so rate_scale plays the analogous role: accuracy stays within about 1 point of 81–82% across rate_scale ∈ {0.3 ... 1.0} (results/rate_scale_results.csv) while spike count and latency both change several-fold: the best evidence in this report that spike-cost knobs can often be tuned almost for free with respect to accuracy, for the right coding scheme.

6. Contribution

Contribution 1: Dataset-Dependent Evaluation of SNN Temporal Coding Schemes

This project runs the same coding-scheme comparison on two datasets with deliberately different pixel-intensity statistics (EMNIST's near-bimodal strokes vs. CIFAR-10's broadly spread natural-image intensities) and shows that both the tuned hyperparameters (section 5.2, section 5.3) and the sensitivity of each scheme to its own hyperparameters change between them, even though the coarse accuracy ranking is preserved.

Contribution 2: Multi-Dimensional Evaluation

Every result in this report is reported as a tuple (accuracy, spike counts, fan-out-weighted energy cost, and decision latency) rather than accuracy alone (section 1.6). This is what makes the section 5.1 finding legible in the first place: by accuracy alone, rate coding simply “wins”; adding the other three axes reveals a genuine three-way trade-off with no dominant scheme.

Contribution 3: Sensitivity / Interaction Analysis

Beyond reporting each scheme's best configuration, this project systematically swept each scheme's own hyperparameters (threshold for TTFS/phase, rate_scale for rate, repeat_cycles for phase) and, for phase coding specifically, ran a full joint grid over its two temporal degrees of freedom rather than sweeping them independently (section 4 Experiments 2–3c, section 5.3). This is what surfaced the (num_bins, cycle_steps) interaction effect, the single most surprising and best-verified result in this report, confirmed by a dedicated neighborhood bug-check before being trusted (section 4 Experiment 3c).

Contribution 4: Alignment Between Research Objective, Dataset, and Metric

The conceptual thread tying the other three contributions together: the appropriate coding scheme cannot be identified by accuracy alone, and it cannot be identified from one dataset and assumed to transfer. The choice depends jointly on the dataset's own characteristics and on which objective (accuracy, energy, latency) the SNN system is being built for. This is the claim this project is best positioned to argue in a written report, and it is directly supported by section 5.1–5.3 rather than asserted on its own.

Supporting Contribution: A Reusable Benchmark Harness

A working benchmark harness (config/phase_grid.py + experiments/run_phase_grid.py + experiments/plot_results.py) where adding a new sweep is a matter of adding entries to a config dictionary rather than writing a bespoke script; most of the sensitivity analyses in section 5.3–5.4 were each a few lines of configuration. This harness, and the fairness/tuning conventions documented in section 1.6 and section 3.1, are reusable infrastructure for whoever extends this project next (section 9).

7. What I Learned / Observations

Observation 1: Accuracy Does Not Tell the Whole Story

The initial framing question was simply “which coding scheme is most accurate,” which section 5.1 answers cleanly (rate). But once spike count, SynOps, and latency are reported alongside accuracy, rate's advantage looks much smaller: it is 7× more expensive than TTFS for an 11-point accuracy gain, and phase gets within 4 points of rate's accuracy at a fraction of the cost while being fastest to decide. Whether that trade is “worth it” depends entirely on the deployment objective, which is precisely the point in section 1.3 and section 6 Contribution 4.

Observation 2: Dataset Statistics Matter More Than the Coding Scheme's Label

The same three coding schemes, same architecture, same training protocol produce a materially different sensitivity picture on EMNIST versus CIFAR-10 (section 5.2): a threshold that is nearly free to tune on EMNIST (flat plateau, section 5.4) becomes the single most important hyperparameter on CIFAR-10 (steep, monotonic falloff). A coding scheme's practical behavior appears to be at least as much a property of the interaction between the scheme and the dataset's pixel statistics as it is a property of the scheme in isolation.

Observation 3: Phase Coding Is Highly Parameter-Sensitive, in a Non-Obvious Way

The single largest effect size found anywhere in this project (section 5.3) is not a difference between coding schemes; it is the ~4-point gap between the best and second-best (num_bins, cycle_steps) cell for phase coding on EMNIST alone. Neither of the two single-axis sweeps that were run first (Experiments 2 and 3) would have found this cell; it only appeared once the two axes were swept jointly. That is a useful general lesson beyond this project: a coding scheme with more than one temporal degree of freedom needs a joint sweep, not two sequential single-axis sweeps, before its sensitivity can be trusted.

Observation 4: Static Datasets May Understate What Temporal Coding Differences Look Like

Both datasets used here (section 3.3) are static images with no inherent temporal structure; all timing is introduced by the encoder itself. The DVS literature reviewed in section 2 (Table 2C) suggests event-based data, where timing is native to the sensor rather than synthetically imposed, may be a fundamentally different regime for evaluating temporal codes. This project's conclusions (section 5, section 6) should be read as specific to static, artificially time-encoded input; whether they hold on genuinely event-based data is an open question this project did not test (section 8, section 9).

8. Limitations

- Only one neuron model was used (single-compartment LIF with a fixed $\tau = 2.0$, $v_{th} = 0.1$); results may not generalize to other neuron models (e.g., adaptive threshold, multi-compartment).
- Only one network architecture was used: a single fully-connected hidden layer (512 units). No convolutional, recurrent, or deeper architecture was tested; section 5.2 explicitly flags this as the likely binding constraint on CIFAR-10 accuracy, ahead of coding-scheme choice.
- Only two datasets, both static images (EMNIST, CIFAR-10); no event-based dataset (e.g., DVS-Gesture, CIFAR10-DVS) was run under this project's methodology, despite being reviewed in the literature (section 2, section 3.3.3) and despite a related, earlier project already having a working DVS-GC loader and preliminary results not yet re-validated under this project's fairness conventions; this is the largest gap between what was reviewed and what was tested here.
- Burst coding, present throughout the literature reviewed (section 2), was not implemented or evaluated under this project's methodology; this report covers Rate, TTFS, and Phase only, though a related, earlier project has a working burst encoder not yet re-validated here.
- Energy was measured with a fan-out-weighted synaptic-operations proxy (avg_synops / avg_energy_cost), not on actual neuromorphic hardware; it is a reasonable relative-cost proxy but not a validated absolute energy number.
- Hyperparameter search was not exhaustive: some sweeps (threshold_ttfs, threshold_phase on EMNIST; the frequency sweep, Experiment 2) were run at a single seed rather than 3, and are flagged as indicative rather than conclusive wherever they appear in section 5.
- The per-scheme hyperparameter spaces are not identical in size or kind across coding schemes (rate has rate_scale; TTFS has threshold and gamma; phase has threshold, num_bins, and cycle_steps);

this is a deliberate fairness choice (section 1.6, each scheme is tuned on its own internal knobs) but means the amount of tuning effort invested was not perfectly equal across schemes, and a more exhaustive tuning pass on any one scheme could shift its position in section 5.1's ranking somewhat.

- Compute constraints limited how many (dataset $\times$ scheme $\times$ hyperparameter $\times$ seed) cells could be run; the joint frequency $\times$ resolution grid (section 5.3) covers only the collision-free region ($\text{cycle_steps} \geq \text{num_bins}$) and only $4 \times 5 = 20$ points per dataset.
- Matched-versus-mismatched encoding/readout pairings were not compared directly. Every reported result uses a scheme-specific matched pairing (coding == readout), so this report cannot determine whether matching itself provides an advantage. This remains an open question for future work.

9. Future Work

Roughly in priority order given what is already built:

9.1 Immediate Next Step: Test Matched vs. Mismatched Readouts

No experiment currently pairs a mismatched coding/readout combination (e.g., rate-encode $\rightarrow$ phase-readout). A future `cross_coding_readout` experiment covering all nine $\{\text{rate, ttfs, phase}\} \times \{\text{rate, ttfs, phase}\}$ combinations across 3 seeds would directly test whether scheme-specific matched readouts provide an advantage over mismatched pairings.

9.2 Datasets

- A genuinely event-based dataset (e.g., DVS-Gesture, CIFAR10-DVS) to test whether section 5's dataset-dependence argument extends from “static vs. static” to “static vs. event-based,” directly motivated by the DVS literature reviewed but not tested under this project's methodology (section 2, section 8). A related, earlier project already has a working DVS-GC loader and preliminary rate/TTFS/phase/burst results on it; porting that loader and re-running it under section 1.6's fairness and matched-encoding-readout conventions is a smaller lift than starting from scratch.
- Larger-scale and additional static datasets (e.g., Fashion-MNIST, CIFAR-100) to test whether the accuracy ranking and hyperparameter-transfer findings in section 5.2 hold beyond two datasets.

9.3 Models

- A convolutional SNN backbone, at least for CIFAR-10, to check whether the coding-scheme ordering (rate > phase > TTFS, section 5.2) holds with more model capacity, or whether a weak backbone is compressing differences between schemes that would show up more clearly with a stronger one.
- Recurrent SNN architectures, as an alternative to the single-hidden-layer design used throughout this report. Transformer/SNN hybrid architectures are noted only as a longer-term, more speculative direction with a weaker direct connection to this project's current scope.

9.4 Coding

- Finish Experiment 8 end-to-end: wire the soft/differentiable frequency relaxation into the main training pipeline as an actual trainable `cycle_steps` config (with discrete snap-back at evaluation), and compare its result against the best swept (`num_bins`, `cycle_steps`) from section 5.3; a positive result would show that learning can replace grid search here without losing accuracy.
- Burst coding, to close the gap between the literature reviewed (section 2) and what has actually been implemented here. A related, earlier project has a working burst encoder; re-validating it under this project's matched-encoding-readout and fairness conventions (section 1.6) is more tractable than implementing one from scratch.
- Hybrid or adaptive coding schemes more broadly: the natural extension of this project's core finding that “instead of selecting a coding scheme manually, an adaptive or dataset-aware coding scheme could automatically select or adjust its encoding strategy according to the temporal characteristics of the input data.” This follows directly from section 5.2's finding that the best hyperparameters (and even the best coding scheme, depending on the objective) are dataset-dependent rather than universal.

9.5 Evaluation

- Real neuromorphic hardware, or at minimum a validated hardware-calibrated energy model, to check whether the `avg_synops` proxy (section 1.6, section 8) actually tracks real energy consumption in the direction and magnitude this report assumes.
- Memory footprint and throughput, alongside the accuracy/energy/latency triad already used, for a more complete deployment-oriented comparison.
- A precise ablation of the phase-readout windowing effect noted in section 4 Experiment 3c, currently a qualitative observation (windowing shifts some grid cells but not others) rather than a characterized effect.